

HW2：骰子模擬器實作

學號：A1123350

姓名：許敬惟

與 AI 對話：

Exercise 1
骰子模擬器
需求：為骰子模擬器，尺寸 400x320，標題為「骰子模擬器」。
骰子模擬器應具備以下功能：
1. 顯示骰子擲出的點數。
2. 顯示骰子擲出的總和。
3. 顯示骰子擲出的平均數。
4. 顯示骰子擲出的次數。
5. 顯示骰子擲出的時間。
6. 顯示骰子擲出的日期。
7. 顯示骰子擲出的時間和日期。
8. 顯示骰子擲出的時間、日期和時間。

要實作這個程式，然後不用製作骰子按鈕的功能，只要有按鈕就好

依需求實作的 Java Swing 程式碼如下，介面具備上方統計標籤、中央 60pt 點數標籤以及下方的「擲骰子」按鈕，且按鈕僅保留純按鈕元件，不包含點擊計算等觸發功能： PNG

Java



```
import javax.swing.*;
import java.awt.*;

public class DiceSimulator extends JFrame {

    public DiceSimulator() {
        // 視窗標題「骰子模擬器」，尺寸 400x320
        setTitle("骰子模擬器");
        setSize(400, 320);

        // 關閉時結束程式，開啟時置中
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        // 使用 BorderLayout 配置版面
        setLayout(new BorderLayout());

        // 視窗上方顯示「已擲 N 次，總和 M，平均 X.XX」
        JLabel topLabel = new JLabel("已擲 0 次，總和 0，平均 0.00", SwingConstants.CENTER);
        topLabel.setPreferredSize(new Dimension(400, 40));
        add(topLabel, BorderLayout.NORTH);
```

```
// 中央 JLabel 顯示目前點數，字體 60pt (預設顯示 -)
JLabel diceLabel = new JLabel("-", SwingConstants.CENTER);
diceLabel.setFont(new Font("SansSerif", Font.BOLD, 60));
add(diceLabel, BorderLayout.CENTER);

// 下方一個「擲骰子」按鈕 (純元件，未加上動作監聽)
JButton rollButton = new JButton("擲骰子");
rollButton.setPreferredSize(new Dimension(400, 45));
add(rollButton, BorderLayout.SOUTH);

// 顯示視窗
setVisible(true);
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new DiceSimulator());
}
}
```

重點設定說明

- 版面配置：採用 `BorderLayout`，將統計文字置於 `NORTH`、中央大字標籤置於 `CENTER`，下方按鈕置於 `SOUTH`。 
- 點數文字大小：設定 `diceLabel.setFont(new Font("SansSerif", Font.BOLD, 60));` 滿足 60pt 字體規格。
○ 
- 按鈕功能：僅建立並配置 `JButton("擲骰子")`，未註冊 `ActionListener`，維持純靜態按鈕元件。 

👍 🗨️ 📄 ...

中間可以做成骰子樣式嗎



👍 🗨️ 📄 ...

沒辦法靠修改程式碼做到嗎

可以完全透過 Java 內建的 `Graphics2D` 自訂繪製來達成，不需要任何外部圖片檔。

以下是改寫後的程式碼：中間建立了一個圓角正方形的骰子方塊，並使用幾何演算法畫出點數圓點（以 1 點為預設展示），下方按鈕一樣保留純按鈕元件：

Java



```
import javax.swing.*.*;
import java.awt.*.*;

public class DiceSimulator extends JFrame {

    public DiceSimulator() {
        // 視窗標題「骰子模擬器」，尺寸 400x320
        setTitle("骰子模擬器");
        setSize(400, 320);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // 上方統計標籤
        JLabel topLabel = new JLabel("已擲 0 次，總和 0，平均 0.00", SwingConstants.CENTER);
        topLabel.setPreferredSize(new Dimension(400, 40));
        add(topLabel, BorderLayout.NORTH);

        // 中間骰子面板（預設顯示 1 點紅色圓點）
        DicePanel dicePanel = new DicePanel(1);
        add(dicePanel, BorderLayout.CENTER);

        // 下方按鈕（純按鈕，不綁定功能）
        JButton rollButton = new JButton("擲骰子");
        rollButton.setPreferredSize(new Dimension(400, 45));
        add(rollButton, BorderLayout.SOUTH);

        setVisible(true);
    }
}
```

```
// 自訂繪製骰子的面板
static class DicePanel extends JPanel {
    private int dots;

    public DicePanel(int dots) {
        this.dots = dots;
        setBackground(new Color(240, 240, 240));
    }

    public void setDots(int dots) {
        this.dots = dots;
        repaint();
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        // 開啟反鋸齒，讓邊角和圓點更平滑
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

        int size = 120; // 骰子方塊尺寸
        int x = (getWidth() - size) / 2;
        int y = (getHeight() - size) / 2;

        // 繪製微立體陰影
        g2.setColor(new Color(200, 200, 200));
        g2.fillRoundRect(x + 4, y + 4, size, size, 28, 28);

        // 繪製骰子本體（白底圓角方塊）
        g2.setColor(Color.WHITE);
        g2.fillRoundRect(x, y, size, size, 28, 28);

        // 繪製骰子外邊框
        g2.setColor(new Color(180, 180, 180));
        g2.setStroke(new BasicStroke(2));
        g2.drawRoundRect(x, y, size, size, 28, 28);

        // 點數圓點直徑
        int dotSize = (dots == 1) ? 26 : 18;

        // 點數 1 為紅色，其他點數為黑色
        g2.setColor(dots == 1 ? new Color(210, 40, 40) : new Color(30, 30, 30));
```

Java

```
// 計算九宮格圓點座標位置
int left = x + 25;
int cx = x + size / 2 - dotSize / 2;
int right = x + size - 25 - dotSize;

int top = y + 25;
int cy = y + size / 2 - dotSize / 2;
int bottom = y + size - 25 - dotSize;

// 依照點數決定各圓點繪製位置
if (dots == 1 || dots == 3 || dots == 5) {
    g2.fillOval(cx, cy, dotSize, dotSize); // 中心點
}
if (dots >= 2) {
    g2.fillOval(left, top, dotSize, dotSize); // 左上
    g2.fillOval(right, bottom, dotSize, dotSize); // 右下
}
if (dots >= 4) {
    g2.fillOval(right, top, dotSize, dotSize); // 右上
    g2.fillOval(left, bottom, dotSize, dotSize); // 左下
}
if (dots == 6) {
    g2.fillOval(left, cy, dotSize, dotSize); // 左中
    g2.fillOval(right, cy, dotSize, dotSize); // 右中
}
}
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new DiceSimulator());
}
}
```

實作特點

1. **純 Java 原生繪圖：**繼承 `JPanel` 並複寫 `paintComponent()`，利用 `Graphics2D` 畫出圓角方塊、外框與立體陰影。
2. **支援 1 到 6 點配置：**內部九宮格座標演算法可支援 1 到 6 點的標準骰子排版。
3. **符合作業視覺規格：**點數 1 點特別使用經典的加大紅色圓點呈現。 

Reflection：我先依照題意請 AI 生成骰子模擬器的程式碼，但我看成果不太符合骰子模擬器的定義，因此我想將中間的數字改成骰子樣式，但 AI 誤解我的意思，所以我請 AI 重新整理程式碼給我。